

N0mn0m.exe

Opening the binary in IDA, we can see there are 0 function imports in the binary and only a handful of functions which could indicate this is some kind of packer for a 2nd stage payload.

[illegible]

Opening it in hexedit, we can see there are signs of UPX packer. Notice the string MRG0, MRG1 and MRG!. This has been intentionally modified from the original header UPX0, UPX1 and UPX!. To undo this, just change the bytes back to UPX from MRG.

```
(kali㉿kali)-[~/Reverse_Engineering_-_M1r4g3/N0mn0M.exe/files/tmp]
$ upx -d N0mn0M.exe
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

  File size      Ratio      Format      Name
  ──────────  ──────────  ──────────  ──────────
    427008 ←    237056    55.52%    win64/pe    N0mn0M.exe

Unpacked 1 file.
```

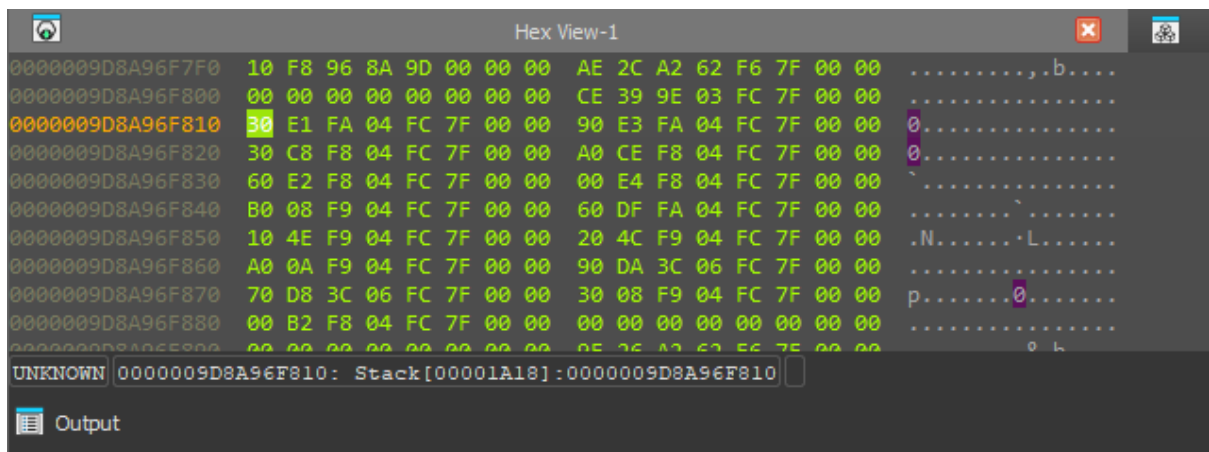
After that, upx should be able to unpack it successfully.

```

0  _DWORD v9[28]; // [rsp+C0h] [rbp-768h] BYREF
1  _QWORD v10[154]; // [rsp+130h] [rbp-6F8h] BYREF
2  _BYTE v11[528]; // [rsp+600h] [rbp-228h] BYREF
3
4  if ( !(__int8)std::_Tree<std::_Tmap_traits<void *,`anonymous namespace'::_Mutex_count_pair,std::less<void *>,std::_Crt_allocator<
5      return 0;
6  v6 = word_7FF662A26010;
7  if ( word_7FF662A26010[0] != 0x5A4D ) // MZ
8      return 0;
9  v4 = (char *)word_7FF662A26010 + *((int *)v6 + 15);
10 if ( *(_DWORD *)v4 != 0x4550 ) // PE
11     return 0;
12 v9[0] = 104;
13 memset(&v9[2], 0, 0x60u);
14 memset(v2, 0, sizeof(v2));
15 (*void (__fastcall **)(_QWORD, _BYTE *, __int64))(a1 + 32)((0, v11, 260);
16 if ( !(*(__int8) (__fastcall **)(_BYTE *, _QWORD, _QWORD, _QWORD, _DWORD, int, _QWORD, _QWORD, _DWORD *, _QWORD *))(a1 + 24))((
17     v11,
18     0,
19     0,
20     0,
21     0,
22     0,
23     0,
24     0,
25     0,
26     0,
27     0,
28     0,
29     0,
30     0,
31     0,
32     0,
33     0,
34     0,
35     0,
36     0,
37     0,
38     0,
39     0,
40     0,
41     0,
42     0,
43     0,
44     0,
45     0,
46     0,
47     0,
48     0,
49     0,
50     0,
51     0,
52     0,
53     0,
54     0,
55     0,
56     0,
57     0,
58     0,
59     0,
60     0,
61     0,
62     0,
63     0,
64     0,
65     0,
66     0,
67     0,
68     0,
69     0,
70     0,
71     0,
72     0,
73     0,
74     0,
75     0,
76     0,
77     0,
78     0,
79     0,
80     0,
81     0,
82     0,
83     0,
84     0,
85     0,
86     0,
87     0,
88     0,
89     0,
90     0,
91     0,
92     0,
93     0,
94     0,
95     0,
96     0,
97     0,
98     0,
99     0,
100    0,
101    0,
102    0,
103    0,
104    0,
105    0,
106    0,
107    0,
108    0,
109    0,
110    0,
111    0,
112    0,
113    0,
114    0,
115    0,
116    0,
117    0,
118    0,
119    0,
120    0,
121    0,
122    0,
123    0,
124    0,
125    0,
126    0,
127    0,
128    0,
129    0,
130    0,
131    0,
132    0,
133    0,
134    0,
135    0,
136    0,
137    0,
138    0,
139    0,
140    0,
141    0,
142    0,
143    0,
144    0,
145    0,
146    0,
147    0,
148    0,
149    0,
150    0,
151    0,
152    0,
153    0,
154    0,
155    0,
156    0,
157    0,
158    0,
159    0,
160    0,
161    0,
162    0,
163    0,
164    0,
165    0,
166    0,
167    0,
168    0,
169    0,
170    0,
171    0,
172    0,
173    0,
174    0,
175    0,
176    0,
177    0,
178    0,
179    0,
180    0,
181    0,
182    0,
183    0,
184    0,
185    0,
186    0,
187    0,
188    0,
189    0,
190    0,
191    0,
192    0,
193    0,
194    0,
195    0,
196    0,
197    0,
198    0,
199    0,
200    0,
201    0,
202    0,
203    0,
204    0,
205    0,
206    0,
207    0,
208    0,
209    0,
210    0,
211    0,
212    0,
213    0,
214    0,
215    0,
216    0,
217    0,
218    0,
219    0,
220    0,
221    0,
222    0,
223    0,
224    0,
225    0,
226    0,
227    0,
228    0,
229    0,
230    0,
231    0,
232    0,
233    0,
234    0,
235    0,
236    0,
237    0,
238    0,
239    0,
240    0,
241    0,
242    0,
243    0,
244    0,
245    0,
246    0,
247    0,
248    0,
249    0,
250    0,
251    0,
252    0,
253    0,
254    0,
255    0,
256    0,
257    0,
258    0,
259    0,
260    0,
261    0,
262    0,
263    0,
264    0,
265    0,
266    0,
267    0,
268    0,
269    0,
270    0,
271    0,
272    0,
273    0,
274    0,
275    0,
276    0,
277    0,
278    0,
279    0,
280    0,
281    0,
282    0,
283    0,
284    0,
285    0,
286    0,
287    0,
288    0,
289    0,
290    0,
291    0,
292    0,
293    0,
294    0,
295    0,
296    0,
297    0,
298    0,
299    0,
300    0,
301    0,
302    0,
303    0,
304    0,
305    0,
306    0,
307    0,
308    0,
309    0,
310    0,
311    0,
312    0,
313    0,
314    0,
315    0,
316    0,
317    0,
318    0,
319    0,
320    0,
321    0,
322    0,
323    0,
324    0,
325    0,
326    0,
327    0,
328    0,
329    0,
330    0,
331    0,
332    0,
333    0,
334    0,
335    0,
336    0,
337    0,
338    0,
339    0,
340    0,
341    0,
342    0,
343    0,
344    0,
345    0,
346    0,
347    0,
348    0,
349    0,
350    0,
351    0,
352    0,
353    0,
354    0,
355    0,
356    0,
357    0,
358    0,
359    0,
360    0,
361    0,
362    0,
363    0,
364    0,
365    0,
366    0,
367    0,
368    0,
369    0,
370    0,
371    0,
372    0,
373    0,
374    0,
375    0,
376    0,
377    0,
378    0,
379    0,
380    0,
381    0,
382    0,
383    0,
384    0,
385    0,
386    0,
387    0,
388    0,
389    0,
390    0,
391    0,
392    0,
393    0,
394    0,
395    0,
396    0,
397    0,
398    0,
399    0,
400    0,
401    0,
402    0,
403    0,
404    0,
405    0,
406    0,
407    0,
408    0,
409    0,
410    0,
411    0,
412    0,
413    0,
414    0,
415    0,
416    0,
417    0,
418    0,
419    0,
420    0,
421    0,
422    0,
423    0,
424    0,
425    0,
426    0,
427    0,
428    0,
429    0,
430    0,
431    0,
432    0,
433    0,
434    0,
435    0,
436    0,
437    0,
438    0,
439    0,
440    0,
441    0,
442    0,
443    0,
444    0,
445    0,
446    0,
447    0,
448    0,
449    0,
450    0,
451    0,
452    0,
453    0,
454    0,
455    0,
456    0,
457    0,
458    0,
459    0,
460    0,
461    0,
462    0,
463    0,
464    0,
465    0,
466    0,
467    0,
468    0,
469    0,
470    0,
471    0,
472    0,
473    0,
474    0,
475    0,
476    0,
477    0,
478    0,
479    0,
480    0,
481    0,
482    0,
483    0,
484    0,
485    0,

```

Opening the new binary in IDA, the program seems to be making a lot of indirect function calls. Its using a1 and adding an offset to it to get the function.



To extract the functions, set a breakpoint right at the call instruction and dump the values in `rax` to see the table of function pointers. `0x9d8a96f810` is the start of our table. After manually reviewing each pointer, we can recreate the table as such :

+0x0	VirtualAllocEx
+0x8	WriteProcessMemory
+0x10	ReadProcessMemory
+0x18	CreateProcessW
+0x20	GetModuleFileNameW
+0x28	ResumeThread
+0x30	GetThreadContext
+0x38	SetThreadContext
+0x40	WaitForSingleObject
+0x48	CloseHandle
+0x50	TerminateProcess
+0x58	NtUnmapViewOfSection
+0x60	NtQueryInformationProcess
+0x80	BASE ADDR OF EXE

```

IDA View-A
Pseudocode-A
Hex View-1
Imports

38 memset(v8, 0, sizeof(v8));
39 v5 = ((__int64 (__fastcall *)(_QWORD, _QWORD, _QWORD *, __int64, _QWORD))a1->NtQueryInformationProcess)(
40     v2[0],
41     0,
42     v8,
43     48,
44     0);
45 if ( !v5
46     && (((void (__fastcall *)(_QWORD, __int64, __int64 *, __int64, _QWORD))a1->ReadProcessMemory)(
47         v2[0],
48         v8[1] + 16LL,
49         &v7,
50         8,
51         0),
52         ((void (__fastcall *)(_QWORD, __int64))a1->NtUnmapViewOfSection)(v2[0], v7),
53         (v3 = sub_7FF662A21030(a1, v2[0], *((_QWORD *)v4 + 6), *((unsigned int *)v4 + 20))) != 0)
54     && (unsigned __int8)sub_7FF662A214B0(a1, v2[0], v3)
55     && (unsigned __int8)sub_7FF662A211C0(a1, v2[0], v3)
56     && (((void (__fastcall *)(_QWORD, __int64, __int64 *, __int64, _QWORD))a1->WriteProcessMemory)(
57         v2[0],
58         v8[1] + 16LL,
59         &v3,

```

In IDA, I created a struct at a1 so that its easier to see what functions are being called now. After reviewing the functions being called, it looks like its doing Process Hollowing. It starts of by getting a path to itself and creating another process of itself in a suspended state before modifying the process memory with WriteProcessMemory().

Task Manager

File Options View

Processes Performance App history Startup Users Details Services

Name	Status	8% CPU	55% Memory	1% Disk	0% Network	Power usage	Power usage t...
Microsoft Distributed Transaction...		0%	0.8 MB	0 MB/s	0 Mbps	Very low	Very low
Microsoft OneDrive		0%	9.7 MB	0 MB/s	0 Mbps	Very low	Very low
Microsoft Software Protection P...		0%	2.2 MB	0 MB/s	0 Mbps	Very low	Very low
Microsoft Text Input Application		0%	7.2 MB	0 MB/s	0 Mbps	Very low	Very low
Microsoft Update Health Service		0%	0.5 MB	0 MB/s	0 Mbps	Very low	Very low
Microsoft Windows Search Filte...		0%	1.0 MB	0 MB/s	0 Mbps	Very low	Very low
Microsoft Windows Search Inde...		0%	9.3 MB	0 MB/s	0 Mbps	Very low	Very low
Microsoft Windows Search Prot...		0%	1.7 MB	0 MB/s	0 Mbps	Very low	Very low
N0mn0M	Suspended	0%	0.4 MB	0 MB/s	0 Mbps	Very low	Very low
N0mn0M	Suspended	0%	0.4 MB	0 MB/s	0 Mbps	Very low	Very low
N0mn0M		0%	0.6 MB	0 MB/s	0 Mbps	Very low	Very low
News and interests (7)		0%	147.8 MB	0 MB/s	0 Mbps	Very low	Very low

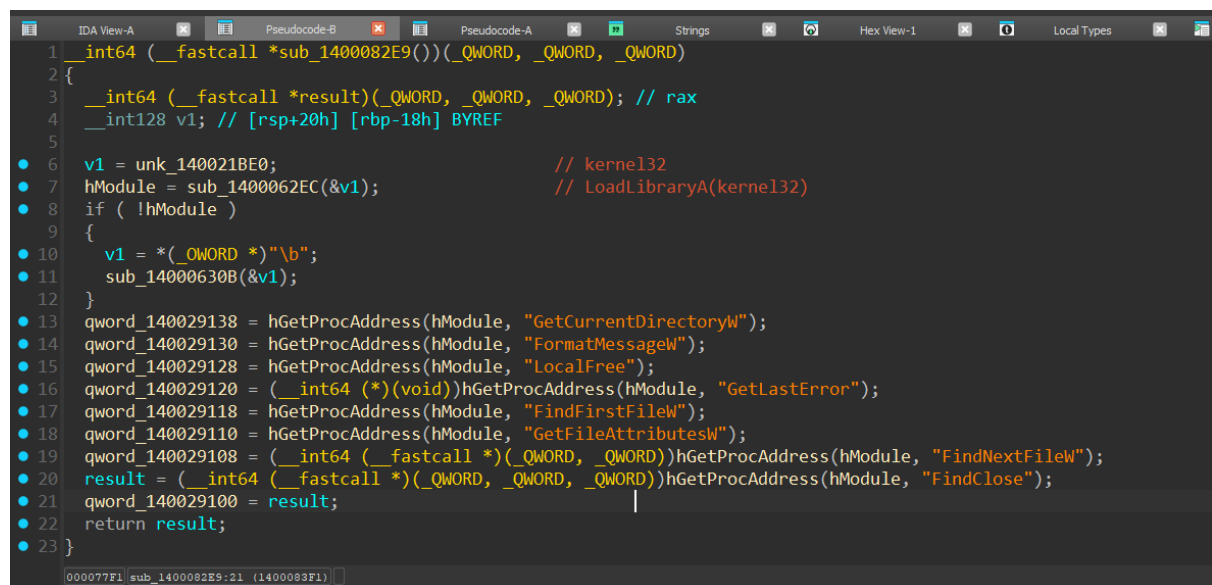
I set a breakpoint at ResumeThread and ran it until it reached the breakpoint. Then, go to Task Manager to find theres another N0mn0M process suspended.

```

PS C:\Users\fyr3p4w\Downloads\pe-sieve64> .\pe-sieve.exe /pid 5760 /dir out
PID: 5760
Output filter: no filter; dump everything (default)
Dump mode: autodetect (default)
[-] Could not set debug privilege
[*] Using raw process!
Scanning workingset: 22 memory regions.
[!] Scanning detached: 00007FFB8170000 : C:\Windows\System32\ntdll.dll
[*] Workingset scanned in 63 ms.
[+] Report dumped to: process_5760
[*] Dumped module to: C:\Users\fyr3p4w\Downloads\pe-sieve64\out\process_5760\140000000.exe as UNMAPPED
[+] Dumped modified to: process_5760
[+] Report dumped to: process_5760
---
PID: 5760
---
SUMMARY:
Total scanned:      1
Skipped:            0
-
Hooked:             0
Replaced:           0
Hdrs Modified:      0
IAT Hooks:          0
Implanted:          1
Implanted PE:       1
Implanted shc:       0
Unreachable files:  0
Other:              0
-
Total suspicious:   1
---
PS C:\Users\fyr3p4w\Downloads\pe-sieve64>

```

Dumped the binary using pe-sieve.



```

1  __int64 (__fastcall *sub_1400082E9)(_QWORD, _QWORD, _QWORD)
2  {
3      __int64 (__fastcall *result)(_QWORD, _QWORD, _QWORD); // rax
4      __int128 v1; // [rsp+20h] [rbp-18h] BYREF
5
6      v1 = unk_140021BE0; // kernel32
7      hModule = sub_1400062EC(&v1); // LoadLibraryA(kernel32)
8      if ( !hModule )
9      {
10         v1 = *(_OWORD *)"\b";
11         sub_14000630B(&v1);
12     }
13     qword_140029138 = hGetProcAddress(hModule, "GetCurrentDirectoryW");
14     qword_140029130 = hGetProcAddress(hModule, "FormatMessageW");
15     qword_140029128 = hGetProcAddress(hModule, "LocalFree");
16     qword_140029120 = (__int64 (*)(void))hGetProcAddress(hModule, "GetLastError");
17     qword_140029118 = hGetProcAddress(hModule, "FindFirstFileW");
18     qword_140029110 = hGetProcAddress(hModule, "GetFileAttributesW");
19     qword_140029108 = (__int64 (__fastcall *)(_QWORD, _QWORD))hGetProcAddress(hModule, "FindNextFileW");
20     result = (__int64 (__fastcall *)(_QWORD, _QWORD, _QWORD))hGetProcAddress(hModule, "FindClose");
21     qword_140029100 = result;
22     return result;
23 }

```

After clicking around the start function, we can find this function which is dynamically resolving Windows APIs.

```

1  __int64 (__fastcall *dynamic_resolve_api())(_QWORD, _QWORD, _QWORD)
2  {
3      __int64 (__fastcall *result)(_QWORD, _QWORD, _QWORD); // rax
4      __int128 v1; // [rsp+20h] [rbp-18h] BYREF
5
6      v1 = unk_140021BE0; // kernel32
7      hModule = sub_1400062EC(&v1); // LoadLibraryA(kernel32)
8      if ( !hModule )
9      {
10         v1 = *(_QWORD *)"\b";
11         sub_14000630B((__int64 *)&v1);
12     }
13     GetCurrentDirectoryW = hGetProcAddress(hModule, "GetCurrentDirectoryW");
14     FormatMessageW = hGetProcAddress(hModule, "FormatMessageW");
15     LocalFree = hGetProcAddress(hModule, "LocalFree");
16     GetLastError_0 = (__int64 (*)(void))hGetProcAddress(hModule, "GetLastError");
17     FindFirstFileW = hGetProcAddress(hModule, "FindFirstFileW");
18     GetFileAttributesW = hGetProcAddress(hModule, "GetFileAttributesW");
19     FindNextFileW = (__int64 (__fastcall *)(_QWORD, _QWORD))hGetProcAddress(hModule, "FindNextFileW");
20     result = (__int64 (__fastcall *)(_QWORD, _QWORD, _QWORD))hGetProcAddress(hModule, "FindClose");
21     FindClose = result;
22     return result;
23 }

```

We can rename them so that its easier to see. Next, after some more clicking, we will stumble on sub_14000BD3F which looks like the main logic area.

```

return sub_140006C8F();
xmmword_140029190 = v0;
v26 = unk_140021F60;
sub_14000A600(&v27, &v26, 0x6AD1D69);
v2 = v27;
if ( !*v1 )

```

Taking a look at the decompilation, sub_14000a600 looks like the nim strenc function which XOR decrypts strings at runtime using a randomly generated key.

```

[+] Loaded 754 symbols
[+] Found 8 references to sub_14000A600 @ 0x14000a600
[1/8] SUCCESS @ 0x140006a39: 'different lengths for slice assignment'
[2/8] SUCCESS @ 0x14000a4f4: 'bcmode.nim(256, 9) `len(iv) == ctx.sizeBlock() ` '
[3/8] SUCCESS @ 0x14000a54d: 'bcmode.nim(257, 9) `nctx.sizeKey() <= len(key) ` '
[4/8] SUCCESS @ 0x14000ae22: 'bcmode.nim(306, 9) `len(input) <= len(output) ` '
[5/8] SUCCESS @ 0x14000ae95: 'bcmode.nim(307, 9) `len(input) mod ctx.sizeBlock() == 0 ` '
[6/8] SUCCESS @ 0x14000b23b: '.M1r4g3'
[7/8] SUCCESS @ 0x14000bda9: '/*.txt'
[8/8] SUCCESS @ 0x14000c01f: '.M1r4g3'

[+] Successfully decrypted 8/8 strings

```

We can write an IDA script to extract all the strings and decrypt them ourselves.

```

129         sub_14000A600(&v27, &v26, 1803897931); // .M1r4g3
130         if ( *v1 )
131             break;
132         v26 = v27;
133         v24 = *((_QWORD *)&v27 + 1);
134         v27 = xmmword_140029180;
135         v16 = sub_140007040((__int64 *)&v27, (__int64 *)&v26); // Check if full path matches marker (likely endsWith ".M1r4g3")
136         if ( *v1 ) // return true if already encrypted
137             break;
138         v17 = v24;
139         if ( !v16 ) // if not encrypted, encrypt
140         {
141             v27 = xmmword_140029180;
142             encrypt((__int64 *)&v27);
143             v17 = v24;
144             if ( *v1 )
145                 break;

```

After analyzing the program further, its getting a list of all files in the directory, skipping files with the name “.” and “..” and checking if the file has “.M1r4g3” extension on them. If they do, then skip the file. Else, encrypt the file.

Once again, we can let GPT do the heavy lifting and write a decryption script for it

```

from Crypto.Cipher import AES
import re

data = open("CYNX_data.txt.M1r4g3","rb").read()
iv_prefix, ct = data[:16], data[16:]
orig_len = 13002

def rol32(x,r): return ((x<<r)|(x>>(32-r))) & 0xFFFFFFFF

def xorshift(st):
    v1=st[3]
    v2=(st[0]^((st[0]<<11)&0xFFFFFFFF))&0xFFFFFFFF
    st[0],st[1],st[2]=st[1],st[2],v1
    v4=(v2>>8)^v2^v1^(v1>>19); v4&=0xFFFFFFFF; st[3]=v4
    t=((v4>>16)^v4)&0xFFFFFFFF; t=(73244475*t)&0xFFFFFFFF; t^=t>>16
    t=(944304983*t)&0xFFFFFFFF; t^=t>>16; return t&0x7FFFFFFF

def init_state(seed):
    seed&=0xFFFFFFFF
    st=[seed^0xF00DABCD, rol32(seed,16), (322376518-seed)&0xFFFFFFFF, (-
1640531535*seed)&0xFFFFFFFF]
    for _ in range(19): xorshift(st)
    return st

def prng1_ks_and_seed(n):
    st=init_state(n); ks=bytearray(n)
    for i in range(n):
        ks[i]=xorshift(st)&0xFF
        st=init_state(xorshift(st))
    return bytes(ks), st[0]&0xFFFFFFFF

def lcg_next(s):
    s=(214013*s+2531011)&0xFFFFFFFF
    return s, (s>>16)&0x7FFF

```

```
ks1, seed = prng1_ks_and_seed(orig_len)

s=seed; r=[]
for _ in range(orig_len): s,v=lcg_next(s); r.append(v)

s,_=lcg_next(s)
key=[(lcg_next(s:=lcg_next(s)[0])[1]&0xFF)^0x26 for _ in range(32)]
iv=[ (lcg_next(s:=lcg_next(s)[0])[1]&0xFF)^0x37 for _ in range(16)]
key, iv = bytes(key), bytes(iv)

pt_padded = AES.new(key, AES.MODE_CBC, iv).decrypt(ct)
pad = pt_padded[-1]
pt1 = pt_padded[:-pad]

X = bytes(b ^ (r[i] & 0xFF) for i,b in enumerate(pt1))
pt = bytes(b ^ ks1[i] for i,b in enumerate(X))

print(re.search(rb"CYNX\[^\]+\]", pt).group(0).decode())
```